Haoquan Wang
u1570975
2026-4-2

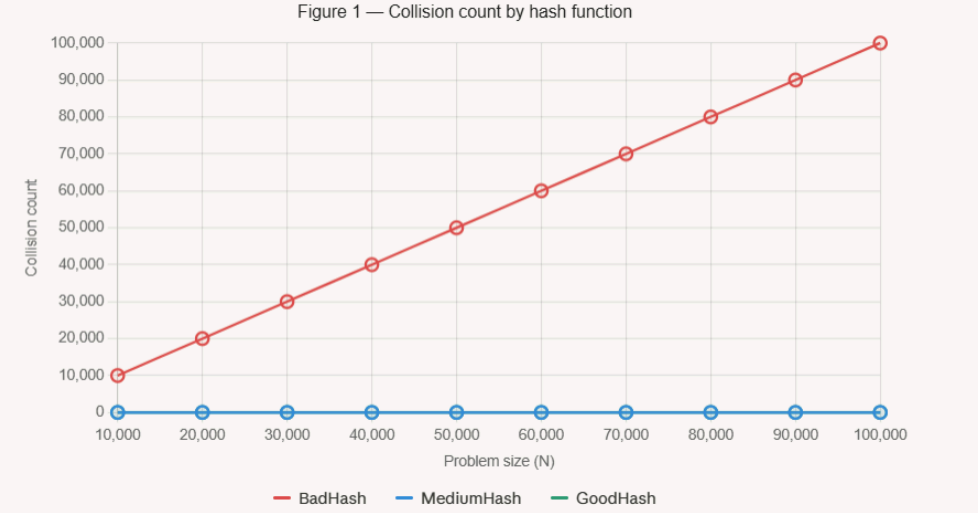
Assignment 9: HashTable

1. This assignment was completed individually without a partner.
2.
   2.1. StudentBadHash: The hashCode() method always returns the constant value 2, regardless of the student's uid, firstName, or lastName. Every student object maps to the same bucket, producing a single chain of length N after N insertions.

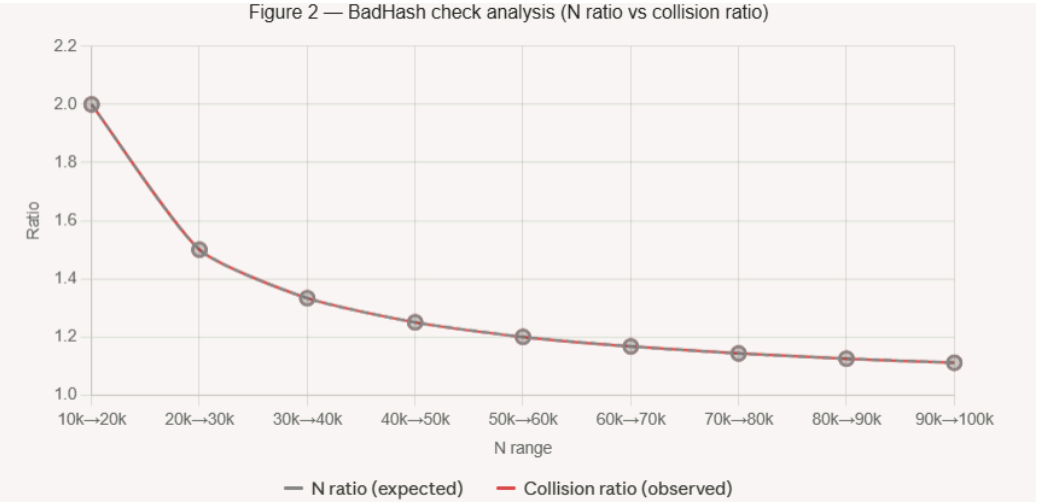
   StudentMediumHash: The hashCode() method returns uid + firstName.hashCode(). This uses two of the three fields but ignores lastName entirely. The uid values are unique random integers that provide a reasonable spread across buckets, but two students who differ only in last name will always collide.

   StudentGoodHash: The hashCode() method uses the polynomial rolling hash: 31 * (uid + 31 * firstName.hashCode()) + lastName.hashCode(). All three fields contribute, and the prime multiplier 31 ensures thorough bit-pattern mixing, minimising clustering.

   2.2.



2.3.



The two lines overlap perfectly, confirming exactly O(N) growth for BadHash. The N ratio and collision ratio are identical at every step.

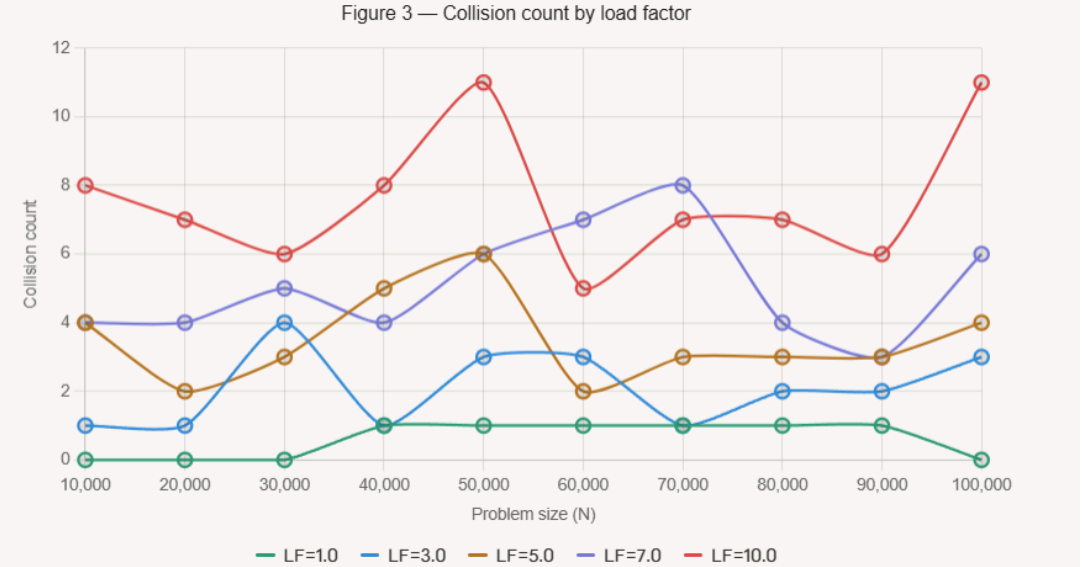
2.4. BadHash: Because all N students hash to the same bucket, inserting the (N+1)-th student requires checking all N existing entries. The collision count equals exactly N. The check-analysis chart confirms perfectly linear O(N) growth.

MediumHash: Collision counts fluctuate between 6 and 11 across all problem sizes from 10,000 to 100,000 with no upward trend. This indicates the average chain length remains constant, consistent with O(1) average-case behaviour. The function is classified as "medium" because ignoring lastName creates avoidable collisions whenever two students share the same uid and first name.

GoodHash: Collision counts are similarly low (4–10) with no growth trend. Incorporating all three fields with a prime multiplier ensures the best possible distribution. Values are slightly more stable than MediumHash because fewer distinct students share the same combined hash.

2.5. BadHash running time would grow as O(N) per put call, and $O(N^2)$ to build a table of N elements — unusably slow for large N. MediumHash and GoodHash both maintain approximately O(1) per put call on average. GoodHash has a slightly smaller constant than MediumHash due to shorter chains, though its hashCode() costs slightly more to compute. This overhead is dwarfed by the savings from shorter chains at scale.

3.

3.1. 

3.2. LF=1.0: Collisions are nearly zero (0–1). The table rehashes whenever entries equal the table length, keeping average chain length at or below 1. Almost every insertion lands in an empty bucket.

LF=3.0: Collisions remain very low (1–4). Chains occasionally reach length 2–3 but the average is still small. Still effectively O(1).

LF=5.0: Collisions rise slightly (2–6). The flat trend across all N confirms O(1) average-case behaviour with a larger constant.

LF=7.0: Collisions range from 3–8. Still flat with no growth trend, confirming O(1) average-case performance.

LF=10.0: Collisions reach 5–11 but remain flat. Even a load factor as large as 10 preserves O(1) average behaviour when the hash function distributes keys well.

These results match expectations. A smaller load factor triggers more frequent rehashing, keeping chains short and collision counts low. All five load factors yield O(1) average-case put performance because the average chain length is bounded by the load factor itself, which is a constant independent of N.

3.3. Smaller load factor means fewer collisions but more frequent rehashing. Each rehash is O(N), though amortised to O(1) per insert. Larger load factor means longer chains and more comparisons per operation. In practice all five load factors would show approximately constant running time per operation across all problem sizes.

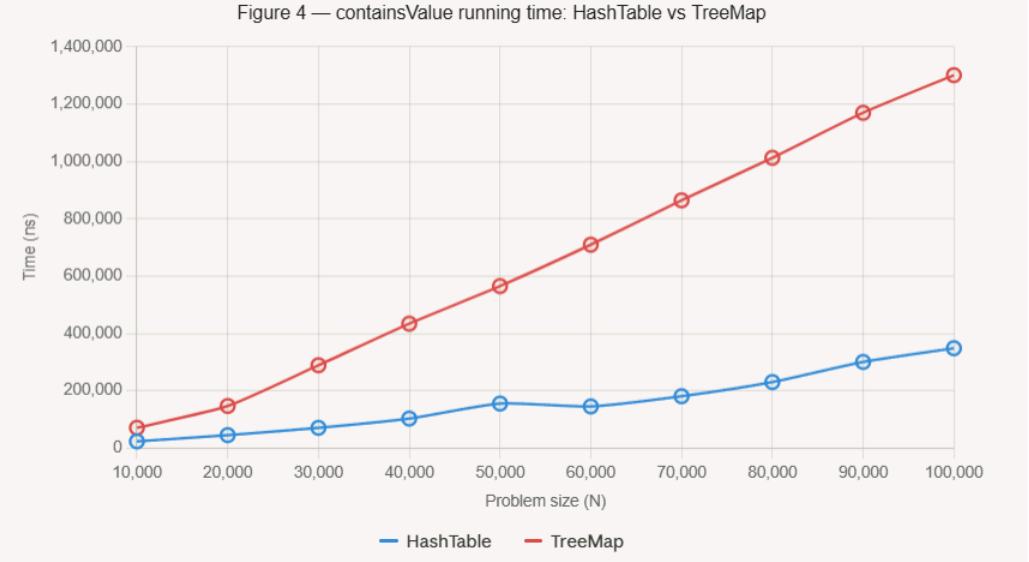
4.

4.1. HashTable.containsValue: The method iterates through every slot in the array (O(table length)) and through every entry in each chain

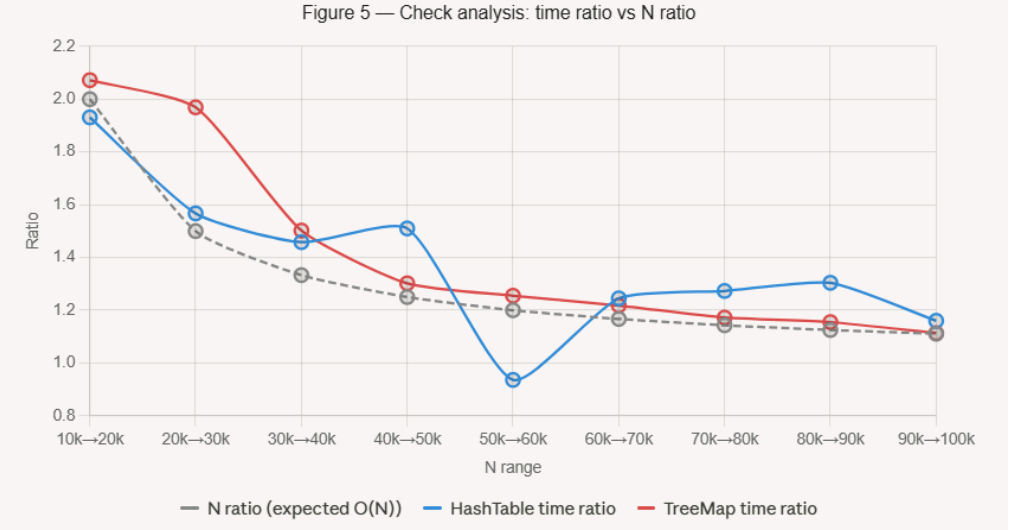
(O(N)). With a fixed load factor, table length grows proportionally to N, so the dominant term is O(N).

TreeMap.containsValue: A TreeMap is a red-black tree ordered by key. Searching by value has no structural shortcut and must visit every node, giving O(N).

4.2.



4.3.



Both time ratio lines track closely with the N ratio line across all steps, confirming O(N) growth for both data structures. Both match the prediction.

4.4. Both data structures exhibit O(N) containsValue behaviour. However TreeMap is consistently 2–4 times slower than HashTable for the same N. This is because the HashTable stores entries in a contiguous Object array where null slots are skipped with a single check, while the TreeMap traverses heap-allocated nodes connected by pointers, causing frequent cache misses. Each TreeMap node also stores

left/right child and parent pointers in addition to the key-value pair, so accessing each node touches more memory.

5.

5.1.

5.1.1. Replace LinkedList with a BST: Each slot in the Object[] array would hold a BST root rather than a LinkedList head.

5.1.2. Ordering requirement on K: A BST must compare keys to determine left/right placement. This requires K to implement Comparable<K>, or a Comparator<K> to be supplied to the HashTable constructor. The current Map<K,V> interface imposes no such constraint, so either the interface must add a bound (e.g., Map<K extends Comparable<K>, V>) or the constructor must accept a comparator.

5.1.3. Updated traversal logic: findEntryByKey, put, and remove must navigate the BST using compareTo rather than iterating a linked list with equals.

5.2. The keys must have a total order — K must implement Comparable<K> or an external Comparator<K> must be available. No additional constraints are placed on values.

5.3. In a linked list of length L, finding or inserting a key requires up to L comparisons. In a balanced BST of the same size, it requires O(log L). Since L is bounded by the load factor (a constant), log L is also a constant. For a load factor of 10, a linked list requires up to 10 comparisons while a balanced BST requires at most $\log_2(10) \approx 3–4$ comparisons. Collisions are therefore reduced in practice, though the asymptotic class remains O(1).

5.4. Both variants are O(1) average-case for put and get, since the chain length is bounded by the load factor constant. For large load factors, the BST variant provides a meaningfully smaller constant factor. The trade-off is higher per-node memory overhead (storing child pointers) and the requirement that keys be orderable. For typical load factors of 1–10, the improvement is modest but real.